

Universidade do Minho

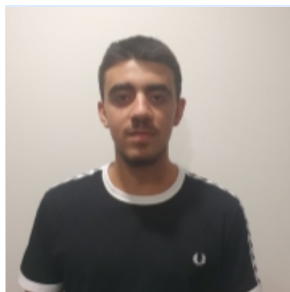
Programação Orientada aos Objetos

Trabalho Prático

Licenciatura em Engenharia Informática - 2022/2023

Grupo 27

João Barroso
A95195



Afonso Bessa
A95225



Francisco Claudino
A89493



Índice

1	Introdução	4
2	Estrutura do Projeto	5
3	Arquitetura de Classes	8
3.1	Artigo	8
3.1.1	Artigo Premium	8
3.1.2	Mala	8
3.1.3	Mala Premium	9
3.1.4	Sapatilha	9
3.1.5	Sapatilha Premium	11
3.1.6	Tshirt	11
3.2	Encomenda	12
3.3	Utilizador	13
3.3.1	CompradorVendedor	13
3.3.2	Transportador	14
3.4	Admin	15
4	Padrão Arquitetural	16
4.1	Model-View-Controller	16
4.2	Model	16
4.3	View	17
4.4	Controller	17
4.5	Interfaces IView, IController e IModel	17
4.6	Comparators	17
4.7	Parser	17
4.8	Exceptions	17
5	Descrição e Funcionamento da Aplicação	18
5.1	CompradorVendedor	20
5.2	Transportador	21
5.3	Admin	21
6	Requisitos	23
6.1	Requisitos Base de Gestão das Entidades	23
6.2	Avançar no Tempo	23
6.3	Efetuar Estatísticas sobre o Estado do Programa	23
6.4	Alterar os Transportadores e prever a noção de Premium	24
6.5	Automatizar a Simulação	24
7	Conclusão	25
8	Referências Bibliográficas / Websites	26

Lista de Figuras

1	Package <i>Source</i> contendo os Packages Artigo e Utilizador	5
2	Package MVC e Package Parser	6
3	Fórmula	9
4	Fórmula	9
5	Fórmula	10
6	Método <i>validarDados()</i>	11
7	Fórmula	11
8	Fórmula	12
9	Menu Inicial	18

10	Menu Comprador Vendedor	18
11	Menu Transportador	18
12	Menu Admin	19
13	Menu Registrar	19
14	Máquina de Estados dos Menus	19
15	Máquina de Estados Comprador Vendedor	20
16	Máquina de Estados do Transportador	21
17	Máquina de Estados do Admin	22

Lista de Tabelas

1	Atributos da Classe Artigos no sistema Vintage	8
2	Atributos da Classe Mala no sistema Vintage	9
3	Atributos da Classe Sapatilha no sistema Vintage	10
4	Atributos da Classe Tshirt no sistema Vintage	12
5	Atributos da Classe Encomenda no sistema.	12
6	Atributos da Classe Utilizador no sistema.	13
7	Atributos da Classe CompradorVendedor no sistema.	14
8	Atributos da Classe Transportador no sistema.	14
9	Atributos da Classe Admin no sistema.	15

1 Introdução

No âmbito da Unidade Curricular de **Programação Orientada aos Objetos** foi proposta a construção de um sistema de *marketplace* chamado **Vintage**. A **Vintage** é um sistema de *marketplace* que possibilita a compra e venda de artigos novos, usados e/ou *premium* de diversos tipos. Neste *marketplace*, os utilizadores registados assumem o papel de comprador, vendedor ou responsável por uma empresa de transporte dos artigos transacionados. Os vendedores podem adicionar novos artigos para venda, enquanto que os compradores têm a opção de adquirir os artigos disponíveis. Por sua vez, os responsáveis pelas empresas transportadoras podem verificar os artigos que precisam de transportar, bem como, alterar a sua fórmula de custo de transporte. Todas as transações de compra e venda são gerenciadas através deste sistema.

Para a conceção deste "mercado" foram utilizados os seguintes *softwares*:

1. **IntelliJ**: um IDE para a linguagem Java. Foi utilizado para escrever o código fonte do sistema, bem como, para realizar testes e depurar erros.
2. **Visual Paradigm**: uma ferramenta de modelagem UML que permitiu criar diagramas de classes. Foi usado para a modelação do sistema antes da implementação, bem como durante a conceção do código fonte, permitindo uma melhor compreensão, estruturação e organização do projeto.

Siglas:

UML - *Unified Modeling Language*

IDE - *Integrated Development Environment*

MVC - *Model-View-Controller*

Keywords:

Java, Vintage, Marketplace

2 Estrutura do Projeto

Após uma leitura aprofundada do enunciado do Trabalho Prático por parte de todo o grupo começamos por retirar conclusões e definir as Classes que seriam necessárias para a construção da *Vintage*.

Depois de analisar cuidadosamente todas secções do Enunciado e com algumas modificações ao longo do desenvolvimento do código fonte, chegamos às seguintes classes e relações / dependências relacionadas com os Utilizadores da *Vintage*, bem como, dos Artigos transacionados nesta mesmo.

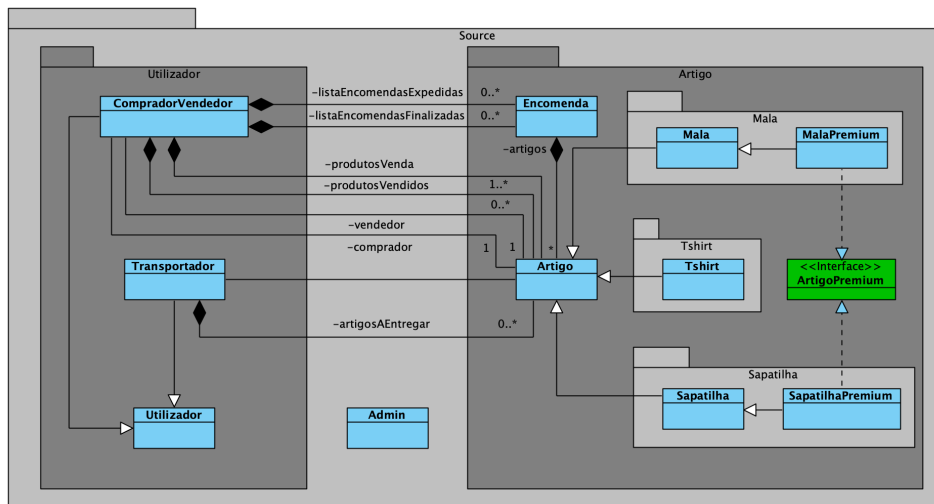


Figura 1: Package *Source* contendo os Packages *Artigo* e *Utilizador*

Como é possível observar, possuímos dois Packages principais, o **Artigo** e o **Utilizador**. O Package *Artigo* possui todas as Classes relacionadas com a informação dos Artigos transacionadas na *Vintage*. Por sua vez, no Package *Utilizador* possuímos todas as classes relacionadas com os vários Utilizadores da *Vintage*.

No Package *Artigo* temos a classe *Artigo*, uma classe abstrata que foi criada com o objetivo de servir de molde para qualquer um dos tipos de artigo implementados neste projeto. As classes *Mala*, *Tshirt* e *Sapatilha* são extensões da classe *Artigo*, cada uma com as suas especificações que serão abordadas com mais detalhe nas suas próprias secções. Por sua vez, surgem também extensões das classes *Mala* - *MalaPremium* - e *Sapatilha* - *SapatilhaPremium* - que representam os Artigos Premium destas classes. Ambas as classes implementam a Interface *ArtigoPremium*. A classe *Encomenda* surge como uma forma de criar uma lista de Artigos, contendo também informações relativas ao tamanho da Embalagem, custo total dos Artigos, bem como, o custo total relativo ao transporte dos Artigos.

Por sua vez, no Package *Utilizador* temos a classe *Utilizador*, uma classe que foi criada com o objetivo de servir de molde para qualquer um dos dois tipos de utilizador disponíveis para utilizar a aplicação. Estes dois utilizadores são os Compradores / Vendedores e os responsáveis pelo transporte dos artigos (Transportadores). Posto isto, neste package encontram-se essas mesmas classes, extensões da classe *Utilizador* e que possuem especificações relativas ao seu funcionamento na aplicação.

Como é possível observar no Diagrama de Classes, a classe do *CompradorVendedor* possui duas listas de encomendas, a lista de encomendas expedidas e a lista de encomendas finalizadas e possui também duas listas de artigos, a lista de artigos vendidos e a lista de

Outra ilação que tiramos do diagrama é que a classe Transportador possui uma lista de artigos relativa aos artigos que possui atualmente para entregar. Por outro lado, podemos observar que as classes MalaPremium e SapatilhaPremium implementam a Interface "ArtigoPremium", algo que abordaremos no seu respectivo capítulo.

```
classDiagram
    package MVC {
        <<Interface>> IModel
        <<Interface>> IView
        <<Interface>> IController
        Model
        View
        Controller
        package Comparators {
            ComparatorComprador
            ComparatorCompradorTempo
            ComparatorVendedor
            ComparatorVendedorTempo
        }
    }
    package Parser {
        Parser
    }
    MVC.Model ..|> MVC.IModel
    MVC.View ..|> MVC.IView
    MVC.Controller ..|> MVC.IController
    MVC.Controller -- MVC.IModel
    MVC.Controller -- MVC.IView
    MVC.Controller -- MVC.IController
    MVC.Comparators.ComparatorComprador -- MVC.IModel
    MVC.Comparators.ComparatorComprador -- MVC.IView
    MVC.Comparators.ComparatorComprador -- MVC.IController
    MVC.Comparators.ComparatorCompradorTempo -- MVC.IModel
    MVC.Comparators.ComparatorCompradorTempo -- MVC.IView
    MVC.Comparators.ComparatorCompradorTempo -- MVC.IController
    MVC.Comparators.ComparatorVendedor -- MVC.IModel
    MVC.Comparators.ComparatorVendedor -- MVC.IView
    MVC.Comparators.ComparatorVendedor -- MVC.IController
    MVC.Comparators.ComparatorVendedorTempo -- MVC.IModel
    MVC.Comparators.ComparatorVendedorTempo -- MVC.IView
    MVC.Comparators.ComparatorVendedorTempo -- MVC.IController
    MVC.Comparators.ComparatorComprador -- MVC.Comparators.ComparatorCompradorTempo
    MVC.Comparators.ComparatorComprador -- MVC.Comparators.ComparatorVendedor
    MVC.Comparators.ComparatorComprador -- MVC.Comparators.ComparatorVendedorTempo
    MVC.Comparators.ComparatorCompradorTempo -- MVC.Comparators.ComparatorVendedor
    MVC.Comparators.ComparatorCompradorTempo -- MVC.Comparators.ComparatorVendedorTempo
    MVC.Comparators.ComparatorVendedor -- MVC.Comparators.ComparatorVendedorTempo
    MVC.Parser.Parser -- MVC.IModel
    MVC.Parser.Parser -- MVC.IView
    MVC.Parser.Parser -- MVC.IController
```

Page 6 of 26

Na figura acima é possível observar a outra metade do Diagrama de Classes, que possui o Package Parser e o Package MVC que contem o Package Comparators.

O Package Parser contem apenas uma classe do mesmo nome e é utilizado para fazer parsing do ficheiro *Logs.txt* onde se encontra uma série de Compradores / Vendedores, Transportadores e Artigos para "povoar" o estado inicial da aplicação.

Por sua vez, o Package Comparator possui quatro classes de Comparadores que são utilizados para construir os *TreeSets* que devolvem algumas das estatísticas que podem ser observadas pelo Admin.

Por fim, temos as classes Model, View e Controller, bem como, as suas interfaces correspondentes IModel, IView e IController que constituem o Padrão Arquitetural escolhido pelo grupo para suportar as funcionalidades da aplicação. Iremos aprofundar este tópico mais à frente.

3 Arquitetura de Classes

3.1 Artigo

A classe abstrata **Artigo** representa um artigo genérico, que contém informações como título, descrição, código, marca, preço, *stock*, Vendedor, Comprador, Transportador, entre outros. Esta classe também define métodos para manipular e obter informações sobre o artigo. Serve como base para a criação de subclasses mais específicas de artigos, que possuem funcionalidades e comportamentos adicionais.

Destaca-se o método abstrato **calcularPreco()**, que é implementado nas subclasses de Artigo. Este método calcula e retorna o preço do artigo com base em regras específicas para cada tipo de artigo.

A classe é declarada como abstrata, o que significa que não pode ser instanciada diretamente, sendo utilizada como base para a criação de subclasses específicas de artigos.

Tabela 1: Atributos da Classe **Artigos** no sistema **Vintage**.

Variável de Instância	Tipo de Dados	Descrição
Título	String	Título do artigo
Descricao	String	Descrição do artigo
Codigo	long	Código alfanumérico único do artigo
Marca	String	Marca do artigo
PrecoBase	double	Preço base do artigo
Stock	int	Quantidade em estoque do artigo
DataEntrega	LocalDate	Data de entrega do artigo
Vendedor	CompradorVendedor	Vendedor do artigo
Comprador	CompradorVendedor	Comprador do artigo
Transportador	Transportador	Transportadora responsável pelo transporte do artigo
Condicao	String	Descrição da condição do artigo
NumeroDonos	int	Número de donos que o artigo teve, caso seja usado
EstadoUtilizacao	double	Avaliação do estado de utilização do artigo

3.1.1 Artigo Premium A *interface* **ArtigoPremium** define um conjunto de funcionalidades que devem ser implementadas por classes que desejam ser consideradas artigos *premium*.

Nesta *interface* estão presentes os seguintes elementos:

1. **calcularPreco()**: Este método é responsável por calcular o preço do artigo premium. A implementação desse método será fornecida pelas classes que implementam **ArtigoPremium**.
2. **valorizacao**: É uma constante que define a valorização atribuída aos artigos *premium*. O seu valor é **5.0f** e pode ser acessada diretamente a partir da *interface* e não pode ser modificada por qualquer classes que a implementam.

3.1.2 Mala A classe **Mala** é uma subclasse da classe abstrata **Artigo** que representa um tipo específico de artigo, neste caso, uma **mala**. Ela adiciona características e comportamentos específicos de uma mala aos atributos e métodos herdados da classe Artigo.

A classe Mala possui as seguintes variáveis de instância adicionais:

Tabela 2: Atributos da Classe **Mala** no sistema **Vintage**.

Variável de Instância	Tipo de Dados	Descrição
Dimensao	double	Dimensão da mala
Material	String	Material da mala
AnoColecao	int	Ano de coleção da mala

O método *calcularPreco()* é uma implementação específica da classe Mala do método abstrato *calcularPreco()* definido na classe Artigo. Esse método calcula e retorna o preço da mala com base na seguinte fórmula:

```

1 public double calcularPreco() {
2     double preco = super.getPrecoBase();
3     if (super.getNumeroDonos() != 0) {
4         preco = preco - (preco / (super.getNumeroDonos() * super.getEstadoUtilizacao() * this.dimensao));
5     }
6     return preco;
7 }

```

Figura 3: Fórmula

3.1.3 Mala Premium A classe **MalaPremium** é uma subclasse da classe Mala que representa uma mala *premium*. Ela implementa a interface **ArtigoPremium** e adiciona recursos específicos de uma mala *premium* aos atributos e métodos herdados.

A classe MalaPremium possui uma variável de instância adicional:

1. **valorizacao**: um valor numérico que representa a valorização específica atribuída à mala *premium*.

Novamente, também esta classe implementa a função *calcularPreco()*, que é uma implementação específica para calcular o preço de uma mala *premium*. Esse cálculo leva em consideração o preço base da mala, a valorização atribuída, o número de donos, o estado de utilização e outras características específicas, tal como é possível ver na figura seguinte:

```

// Função de Cálculo de Preço
1 public double calcularPreco () {
2     double precoFinal = super.getPrecoBase();
3     double extra = super.getAnoColecao() * this.valorizacao;
4     precoFinal = precoFinal + extra;
5     if (super.getNumeroDonos() != 0) {
6         precoFinal = precoFinal - (precoFinal /
7             (super.getNumeroDonos() * super.getEstadoUtilizacao() * super.getDimensao()));
8     }
9     return precoFinal;
10 }

```

Figura 4: Fórmula

3.1.4 Sapatilha A classe **Sapatilha** é uma subclasse da classe abstrata **Artigo** que representa um tipo específico de artigo, neste caso, uma *sapatilha*. Ela adiciona características e

comportamentos específicos de uma sapatilha aos atributos e métodos já herdados da classe Artigo.

A classe Sapatilha possui as seguintes variáveis de instância adicionais:

Tabela 3: Atributos da Classe **Sapatilha** no sistema **Vintage**.

Variável de Instância	Tipo de Dados	Descrição
Tamanho	double	Tamanho da sapatilha
Atacadores	boolean	Indica se a sapatilha possui atacadores
Cor	Color	Cor da sapatilha
DataLancamento	LocalDate	Data de lançamento da sapatilha

A classe também implementa os seguintes métodos:

1. **calcularPreco()**, que é uma implementação específica para calcular o preço de uma sapatilha. Esse cálculo leva em consideração o preço base da sapatilha, o ano de lançamento, o tamanho, o número de donos e o estado de utilização. A função retorna o preço final da sapatilha.

```
// Função de Cálculo de Preço
public double calcularPreco() {
    double precoFinal = super.getPrecoBase();
    if (this.dataLancamento.getYear() != LocalDate.now().getYear() && this.tamanho > 45
        && super.getNumeroDonos() != 0 && super.getEstadoUtilizacao() != 0) {
        precoFinal = precoFinal -
            (precoFinal / (super.getNumeroDonos() * super.getEstadoUtilizacao()));
    }
    return precoFinal;
}
```

Figura 5: Fórmula

2. **validarDados()**, que tem a finalidade de verificar se os dados fornecidos para uma sapatilha são válidos. Essa função realiza duas verificações cruciais:

- (a) **Verificação do tamanho da Sapatilha:** A função assegura que o tamanho da sapatilha esteja dentro de um intervalo adequado. Caso o tamanho seja inferior a 34 ou superior a 47, a função retornará *false*, indicando que os dados da sapatilha não são válidos.
- (b) **Verificação da Cor da Sapatilha:** A função confirma se a cor da sapatilha foi definida corretamente. Se a cor for nula, a função retornará *false*, sinalizando que os dados da sapatilha não são válidos.

Além disso, a função utiliza o método *Color.getColor(cor.toString())* para validar se a cor é um valor válido da classe *Color*. Caso ocorra uma exceção do tipo *IllegalArgumentException* durante essa verificação, a função também retornará *false*, indicando que os dados da sapatilha não são válidos.

```
// Função de Validação de Dados
public boolean validarDados() throws DadosInvalidos {
    if (tamanho < 34 || tamanho > 47) {
        throw new DadosInvalidos( mensagem: "O tamanho está fora do intervalo permitido.");
    }
    if (cor == null) {
        throw new DadosInvalidos( mensagem: "A cor não foi especificada.");
    }
    try {
        Color.getColor(cor.toString());
    } catch (IllegalArgumentException e) {
        throw new DadosInvalidos( mensagem: "A cor especificada é inválida.");
    }
    return true;
}
```

Figura 6: Método *validarDados()*

3.1.5 Sapatilha Premium A classe **Sapatilha Premium** é uma subclasse da classe **Sapatilha** que representa uma sapatilha *premium*. Ela implementa a interface **ArtigoPremium** e adiciona recursos específicos de uma sapatilha *premium* aos atributos e métodos herdados.

A classe **SapatilhaPremium** possui uma variável de instância adicional:

1. **valorizacao**: um valor numérico que representa a valorização específica atribuída à mala *premium*.

Novamente, também esta classe implementa a função *calcularPreco()*, que é uma implementação específica para calcular o preço de uma sapatilha *premium*. Esse cálculo leva em consideração uma fórmula, tal como é possível ver na figura seguinte:

```
// Função de Cálculo de Preço
public double calcularPreco() {
    double precoFinal = super.getPrecoBase();
    double extra = this.valorizacao * 0.5;
    if (super.getDataLancamento().getYear() != LocalDate.now().getYear() && super.getTamanho() > 45
        && super.getNumeroDonos() != 0 && super.getEstadoUtilizacao() != 0) {
        precoFinal = precoFinal -
            (precoFinal / (super.getNumeroDonos() * super.getEstadoUtilizacao()));
    }
    precoFinal = precoFinal + extra;
    return precoFinal;
}
```

Figura 7: Fórmula

3.1.6 Tshirt A classe **Tshirt** é uma subclasse da classe **Artigo** que representa uma **t-shirt**. Esta possui atributos adicionais, como **tamanho**, **padrao** e **desconto**, e implementa métodos específicos para calcular o preço e outras funcionalidades.

A classe **Tshirt** possui os seguintes atributos:

Tabela 4: Atributos da Classe **Tshirt** no sistema **Vintage**.

Variável de Instância	Tipo de Dados	Descrição
Tamanho	enum Tamanho	Tamanho da t-shirt (Valores possíveis: S, M, L, XL)
Padrao	enum Padrao	Padrão da t-shirt (Valores possíveis: LISO, RISCAS, PALMEIRAS)
Desconto	int	Desconto aplicado à t-shirt em porcentagem

Um método importante da classe é o *calcularPreco()*, que calcula o preço da t-shirt com base em determinadas condições.

```
// Função de Cálculo do Preço
public double calcularPreco() {
    double precoFinal = getPrecoBase();
    if (this.padrao != Padrao.LISO && super.getNumeroDonos() != 0) {
        precoFinal = getPrecoBase() - (getPrecoBase() * (1 - (this.desconto * 0.01)));
    }
    return precoFinal;
}
```

Figura 8: Fórmula

3.2 Encomenda

A classe **Encomenda** representa uma encomenda de artigos no sistema. Ela é responsável por armazenar informações sobre os artigos da encomenda, tipo de embalagem, preço final, custos de expedição, estado da encomenda e datas relacionadas.

A classe possui os seguintes atributos:

Tabela 5: Atributos da Classe **Encomenda** no sistema.

Variável de Instância	Tipo de Dados	Descrição
artigos	List<Artigo>	Lista de artigos da encomenda
embalagem	Embalagem	Tipo de embalagem da encomenda (GRANDE, MÉDIO, PEQUENO)
precoFinal	double	Preço final da encomenda
custosExpedicao	double	Custos de expedição da encomenda
estado	Estado	Estado atual da encomenda (PENDENTE, FINALIZADA, EXPEDIDA)
dataCriacao	LocalDate	Data de criação da encomenda
dataExpedicao	LocalDate	Data de expedição da encomenda
dataEntrega	LocalDate	Data prevista de entrega da encomenda

Também possui os seguintes métodos:

1. *devolverEncomenda()*: Verifica se a encomenda está no estado "EXPEDIDA" e se a data atual está antes da data de entrega prevista. Se ambas as condições forem atendidas, cria uma nova encomenda com os mesmos valores da encomenda atual, exceto

pelo estado que é definido como "FINALIZADA". Retorna a nova encomenda, caso contrário, retorna *null*.

2. **adicionarArtigo(Artigo artigo)**: Adiciona um novo artigo à lista de artigos da encomenda. O artigo fornecido é clonado e adicionado à lista.
3. **calcularPrecoFinal()**: Calcula o preço final da encomenda com base nos preços individuais de cada artigo e nas taxas definidas para artigos novos e usados. Percorre a lista de artigos, soma os preços individuais e aplica a taxa correspondente com base no número de donos do artigo. Retorna o preço final calculado.
4. **calcularCustosExpedicao()**: Calcula os custos de expedição da encomenda com base no número de artigos e nos transportadores envolvidos. Utiliza um mapa para contar o número de ocorrências de cada transportador na lista de artigos. Com base no número de ocorrências, calcula os custos de expedição correspondentes, aplicando uma porcentagem sobre o custo total de entrega do transportador. Retorna o valor dos custos de expedição calculados.
5. **atualizaEmbalagem()**: Atualiza o tipo de embalagem da encomenda com base no número de artigos. Se houver apenas um artigo, a embalagem é definida como "PEQUENO". Se houver entre 2 e 4 artigos, a embalagem é definida como "MÉDIO". Caso contrário, a embalagem é definida como "GRANDE". Retorna o novo tipo de embalagem.

Esses métodos permitem manipular e calcular informações relevantes da encomenda, como adicionar artigos, calcular preços e custos, atualizar a embalagem e controlar o processo de devolução da encomenda.

3.3 Utilizador

Essa classe é utilizada para representar e gerenciar as informações de utilizadores no sistema. Agora, segue a tabela dos atributos da classe **Utilizador**:

Tabela 6: Atributos da Classe **Utilizador** no sistema.

Variável de Instância	Tipo de Dados	Descrição
password	String	Senha do utilizador
email	String	Email do utilizador
nome	String	Nome do utilizador
morada	String	Morada do utilizador
numeroFiscal	int	Número fiscal do utilizador

Esses atributos são utilizados para armazenar informações pessoais do utilizador, como senha, email, nome, morada e número fiscal. Eles permitem identificar e autenticar o utilizador no sistema.

3.3.1 CompradorVendedor A classe **CompradorVendedor** representa um Utilizador que atua tanto como **Comprador** quanto como **Vendedor** no sistema. Ela herda da classe Utilizador e adiciona atributos e métodos específicos para essas funcionalidades.

Aqui está a tabela dos atributos da classe **CompradorVendedor**:

No geral, a classe **CompradorVendedor** permite que os **Utilizadores** atuem como **Compradores** e **Vendedores** no sistema, gerenciando seus carrinhos de compras, enco-

Tabela 7: Atributos da Classe **CompradorVendedor** no sistema.

Variável de Instância	Tipo de Dados	Descrição
carrinhoDeCompras	Encomenda	Carrinho de compras do utilizador
listaEncomendasFinalizada	List<Encomenda>	Lista de encomendas finalizadas
listaEncomendasExpedida	List<Encomenda>	Lista de encomendas expedidas
valorTotalCompras	double	Valor total das compras
valorTotalComprasTempo	double	Valor total das compras ao longo do tempo
listaDeFaturasComprador	List<String>	Lista de faturas do comprador
listaDeFaturasVendedor	List<String>	Lista de faturas do vendedor
artigosVendidosData	Map<LocalDate,Artigo>	Mapeamento de datas para artigos vendidos
produtosVendidos	List<Artigo>	Lista de produtos vendidos
produtosVenda	List<Artigo>	Lista de produtos à venda
valorTotalVendas	double	Valor total das vendas
valorTotalVendasTempo	double	Valor total das vendas ao longo do tempo

mendas, faturas, produtos vendidos e valor total de compras e vendas. Com essa funcionalidade, eles podem realizar transações, controlar seus históricos e administrar suas atividades de compra e venda de forma eficiente.

3.3.2 Transportador A classe **Transportador** representa um transportador no sistema. Ela herda da classe **Utilizador** e adiciona atributos e métodos específicos para o transporte de artigos.

Aqui está a tabela dos atributos da classe **Transportador**:

Tabela 8: Atributos da Classe **Transportador** no sistema.

Variável de Instância	Tipo de Dados	Descrição
valorBaseExpedicaoPequena	double	Valor base de expedição para embalagens pequenas
valorBaseExpedicaoMedia	double	Valor base de expedição para embalagens médias
valorBaseExpedicaoGrande	double	Valor base de expedição para embalagens grandes
fatorMultiplicativoImpostos	double	Fator multiplicativo de impostos
margemLucro	double	Margem de lucro
transportePremium	boolean	Indica se o transporte é premium
formula	String	Fórmula para cálculo de custo
artigosAEntregar	List<Artigo>	Lista de artigos a entregar
precoTotalEntregas	double	Preço total das entregas

A classe também possui métodos adicionais para calcular o custo de transporte de acordo com o tamanho da embalagem. Esses métodos utilizam uma fórmula configurável, que leva em consideração o valor base de expedição, a margem de lucro e o fator multiplicativo de impostos. Os métodos ***calculateCostGrande()***, ***calculateCostMedio()*** e ***calculateCostPequeno()*** são responsáveis por realizar esses cálculos para embalagens grandes, médias e pequenas, respectivamente.

3.4 Admin

A classe Admin representa um administrador no sistema. Ela possui atributos como **nome**, **email** e **password**, que armazenam as informações básicas do administrador.

Tabela 9: Atributos da Classe **Admin** no sistema.

Variável de Instância	Tipo de Dados	Descrição
nome	String	Nome do administrador
email	String	Email do administrador
password	String	Senha do administrador

4 Padrão Arquitetural

4.1 Model-View-Controller

A escolha de utilizar o padrão arquitetural **Model-View-Controller (MVC)** no nosso trabalho baseia-se em diversos benefícios que este padrão traz para o desenvolvimento de aplicações, como a **Vintage**.

Uma das principais justificações para a utilização do **MVC** reside na clara separação de responsabilidades que este padrão proporciona. Através da divisão em três componentes distintos:

1. **Model**
2. **View**
3. **Controller**

Foi possível estabelecer uma organização estruturada do código, onde cada componente desempenha um papel específico. Esta separação permite que trabalhem de forma mais eficiente, concentrando-nos em áreas específicas do sistema sem interferir diretamente nas restantes partes. Isso facilita a manutenção e evolução do *software*, pois podemos efetuar modificações num componente sem afetar negativamente o funcionamento dos restantes.

Outra vantagem importante do **MVC** é a reutilização de código. Com a clara divisão de responsabilidades, podemos criar componentes independentes e modularizados. Esta abordagem permite-nos aproveitar e reutilizar esses componentes em diferentes partes da aplicação, poupando tempo e esforço no desenvolvimento. Além disso, a reutilização de código também traz benefícios em termos de manutenção, uma vez que as alterações podem ser aplicadas num componente centralizado e propagadas para todas as partes que o utilizam.

A escalabilidade é outro aspeto relevante proporcionado pelo **MVC**. O padrão arquitetural permite que adicionemos novas funcionalidades ou modifiquemos a interface do utilizador sem afetar diretamente a lógica de negócio subjacente. Isso dá-nos flexibilidade para expandir o sistema ao longo do tempo, adaptando-o às necessidades dos utilizadores e do mercado, sem comprometer a estabilidade e a integridade do código existente.

Embora existam outros padrões arquiteturais disponíveis, a escolha do **MVC** para o nosso trabalho deve-se à sua simplicidade e eficiência comprovada. A sua estrutura bem definida e os benefícios em termos de separação de preocupações, reutilização de código e escalabilidade tornam o **MVC** uma opção sólida e confiável para o desenvolvimento do nosso sistema.

4.2 Model

A classe **Model**, tem como variáveis de instância uma lista de Artigos que corresponde ao Catálogo de Artigos disponíveis para compra, um Map de <String,CompradorVendedor> que corresponde a uma lista de Compradores /Vendedores registados na aplicação, duas listas de Transportadores, uma para os Transportadores Standard e outra para os Transportadores Premium, uma lista de Admin que corresponde aos Admins da aplicação, um double que corresponde ao Lucro da Vintage e um LocalDate que corresponde à data atual da aplicação. Esta classe serve para guardar o estado do programa, como uma espécie de "Base de Dados" onde se tem informação relevante sobre todos os dados necessários ao funcionamento da Vintage.

É utilizado o Map para armazenar a informação relativa aos Compradores / Vendedores pois em várias instâncias trabalhamos com o CompradorVendedor atual como variável, sendo mais fácil e conveniente ter um Map para procurar informação de forma mais eficiente. A chave utilizada neste Map é o email dos Compradores / Vendedores, sendo algo único.

Quanto ao resto da informação, é armazenada nas listas previamente descritas devido ao facto de serem percorridas muitas vezes (não sendo correto percorrer *Maps* mas sim Listas) e de efetuarmos procura por índice, ou seja, o *user* escolhe um dígito de 1 até N e simplesmente vamos à lista e obtemos a opção-1, nas mesmas, levando assim à escolha da *List* como estrutura.

4.3 View

A classe **View** faz parte desta implementação do MVC, sendo que, esta é a que faz toda a interação com o Utilizador, apresentado os menus/mensagens necessárias para obter uma interface gráfica (em modo texto). Os inputs do utilizador são enviados posteriormente para a classe **Controller** que trata de recolher a informação necessária do **Model** e enviar novamente para a **View**.

4.4 Controller

A classe **Controller** é uma classe de ligação entre a classe **Model** e a classe **Controller**, sendo que serve para passar informação do **Model** requerida pela **View**.

4.5 Interfaces IView, IController e IModel

De forma a facilitar a implementação do MVC foram criadas estas interfaces com o objetivo de exportar uma API de uso fácil e simples com o intuito de, por exemplo, se for necessário implementar várias *view*'s diferentes, é apenas preciso que estas implementem a interfaces *IViewer*, sendo as alterações nas restantes classes do MVC mínimas.

4.6 Comparators

Os Comparators implementados são utilizados para efetuar algumas estatísticas relacionadas com a Ordenação dos Compradores / Vendedores em relação á compra / venda de produtos. São utilizados para formar os *TreeSet* que nos fornece essa ordenação. Abordaremos melhor as estatísticas e estas classes na secção destinada ás estatísticas.

4.7 Parser

A classe **Parser** é utilizada para ler o ficheiro *Logs.txt*, efetuar o parsing da informação que este ficheiro contém (Informação relativa a Compradores / Vendedores, Transportadores e Artigos), armazenando a informação no Model aquando do início da aplicação, de forma a iniciar-se a aplicação com dados suficientes para se poder demonstrar todas as funcionalidades da mesma.

4.8 Exceptions

O nosso sistema conta com poucas *exceptions* implementadas, uma vez que o controlo de erros foi efetuado de maneira a que se o *user* imprimir e/ou seleccionar uma opção errada o programa não parar, mas sim tornar a fornecer nova opção escolha.

Dessa maneira, criamos três exceções: uma para o controlo de inputs diferentes do pedido, por exemplo, pedir um inteiro e o *user* fornecer uma string; outra *exception* para o controlo de fórmulas inválidas no momento de criar uma fórmula nova por parte do **Transportador** e, finalmente, uma para verificação de dados válidos na classe **Sapatilhas**.

5 Descrição e Funcionamento da Aplicação

Nesta secção iremos fazer uma breve descrição da aplicação desenvolvida com recurso a diversas ilustrações dos menus desenvolvidos.

Tal como é sugerido no enunciado o email deve ser usado como *login*, dessa maneira, o grupo decidiu criar um menu para seleccionar **Iniciar Sessão**, ou seja, já tem conta criada ou **Registo**, caso pretenda criar uma conta no sistema.

```
+-----+
| Vintage                               Data : 2023-05-14 |
+-----+
| 1 -> Iniciar Sessão                    |
| 2 -> Registar                         |
| 0 -> Sair                             |
+-----+
Opção > |
```

Figura 9: Menu Inicial

Caso a opção seja **Iniciar Sessão**, será pedido o respetivo *email* e *password* associada à conta. Após validado o acesso um dos seguintes três menus irão aparecer:

1. CompradorVendedor

```
+-----+
| CompradorVendedor : Antônio           Data : 2023-05-14
+-----+
| Comprador                             |
+-----+
| 1 -> Ver e/ou Adicionar Artigos       |
| 2 -> Consultar o seu Carrinho de Compras |
| 3 -> Consultar as Encomendas em Exportação |
| 4 -> Devolver uma Encomenda           |
+-----+
| Vendedor                             |
+-----+
| 5 -> Consultar os seus Produtos à venda |
| 6 -> Adicionar Artigo para Venda ou Adicionar Stock |
| 7 -> Remover Artigo do Anúncio         |
+-----+
| 8 -> Consultar Faturas                 |
| 9 -> Avançar no Tempo                  |
| 0 -> Terminar Sessão                 |
+-----+
Opção >
```

Figura 10: Menu Comprador Vendedor

2. Transportador

```
+-----+
| Transportador CTT                     Data : 2023-05-14
+-----+
| 1 -> Consultar artigos por entregar    |
| 2 -> Alterar Valores da Fórmula        |
| 3 -> Alterar Fórmula                   |
| 0 -> Terminar Sessão                 |
+-----+
Opção >
```

Figura 11: Menu Transportador

3. Admin

```
-----+-----
| Admin : Nestor          Data : 2023-05-14 |
|-----+-----
| 1 -> Qual é o Vendedor que mais faturou desde sempre. |
| 2 -> Qual é o Vendedor que mais faturou num determinado período. |
| 3 -> Qual é o Transportador com maior volume de faturação. |
| 4 -> Listar os Artigos vendidos por um Vendedor. |
| 5 -> Obter uma ordenação dos maiores Compradores desde sempre. |
| 6 -> Obter uma ordenação dos maiores Compradores num determinado período. |
| 7 -> Obter uma ordenação dos maiores Vendedores desde sempre. |
| 8 -> Obter uma ordenação dos maiores Vendedores num determinado período. |
| 9 -> Determinar quanto dinheiro ganhou a Vintage no seu funcionamento. |
| 0 -> Terminar Sessão. |
|-----+-----
Opção > |
```

Figura 12: Menu Admin

Caso a opção seja **Registrar**, um novo menu irá aparecer de modo a perguntar ao *user* qual dos dois utilizadores pretende criar.

```
+-----+
| Utilizador              Data : 2023-05-14 |
+-----+
| 1 -> CompradorVendedor |
| 2 -> Transportador     |
| 0 -> Sair              |
+-----+
Opção >
```

Figura 13: Menu Registrar

De seguida, o *user* é obrigado a preencher várias respetivas informações para cada um dos Utilizadores, tal como, foi falado anteriormente na descrição de cada uma das classes.

Em suma, a seguinte **Máquina de Estados** resume os menus iniciais do nosso sistema.

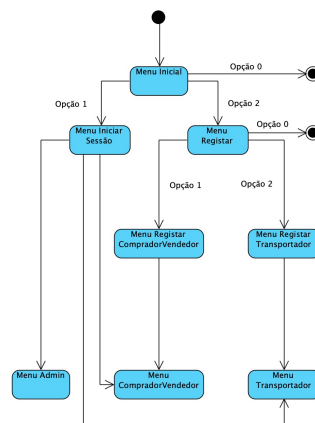


Figura 14: Máquina de Estados dos Menus

5.1 CompradorVendedor

No menu **CompradorVendedor** o *user* tem ao seu dispor uma variedade de opções às quais podemos partir em três pontos:

– Comprador

1. Ver o catálogo de Artigos à venda e/ou adicionar Artigos ao Carrinho de Compras, selecionando a opção 1;
2. Consultar o seu Carrinho de Compras, selecionando a opção 2 e dentro desse mesmo menu Finalizar e/ou Remover Artigos no Carrinho de Compras;
3. Consultar as Encomendas em Exportação, selecionando a opção 3;
4. Devolver uma Encomenda, caso esta esteja ainda a menos de quarenta e oito horas da entrega estar finalizada, selecionando a opção 4.

– Vendedor

1. Consultar todos os seus Produtos à venda, selecionando a opção 5;
2. Adicionar um Artigo novo para a Venda, ou seja, adicionar Mala, Sapatilha ou Tshirt ou Adicionar Stock a um Artigo já à venda, selecionando a opção 6;
3. Remover um Artigo que já esteja à venda podendo selecionar também a quantidade de *stock* a eliminar, selecionando a opção 7;

– Ambos

1. Consultar as Faturas tanto de Vendedor como de Comprador, selecionando a opção 8;
2. Avançar o tempo do Sistema para a data que desejar, selecionando a opção 9;
3. Terminar a sessão e consequentemente gravar o estado do sistema, selecionando a opção 0;

Toda esta informação referida anteriormente pode ser ilustrada através da seguinte figura:

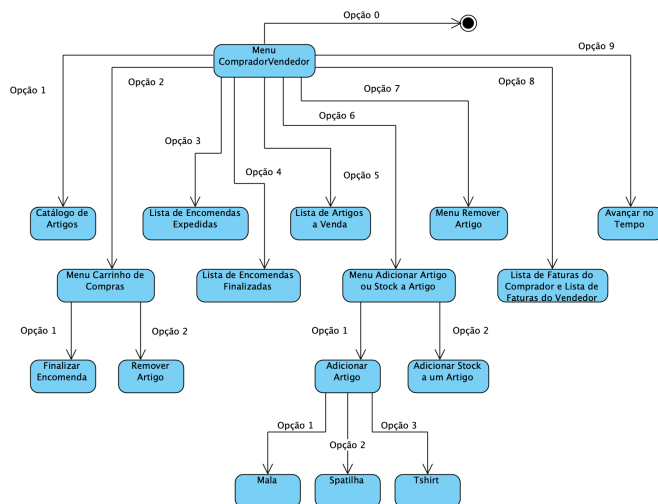


Figura 15: Máquina de Estados Comprador Vendedor

5.2 Transportador

No menu **Transportador** o *user* tem ao seu dispor uma variedade de opções, podendo:

1. Consultar Artigos por Entregar, seleccionando a opção 1;
2. Alterar todos os valores respetivos à fórmula, como, por exemplo, a margem de lucro, seleccionando a opção 2;
3. Alterar por completo a fórmula de cálculo, seleccionando a opção 3;
4. Terminar a sessão e consequentemente gravar o estado do sistema, seleccionando a opção 0;

Toda esta informação referida anteriormente pode ser ilustrada através da seguinte figura:

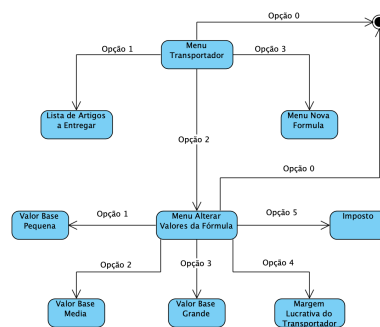


Figura 16: Máquina de Estados do Transportador

5.3 Admin

No menu **Transportador** o *user* tem ao seu dispor uma variedade de opções de estatísticas, podendo:

1. Seleccionar o Vendedor que mais faturou de sempre, seleccionando a opção 1;
2. Seleccionar o Vendedor que mais faturou num determinado período de tempo, seleccionando a opção 2;
3. Seleccionar o Transportador com o maior volume de faturação, seleccionando a opção 3;
4. Listar os Artigos vendidos por um Vendedor, seleccionando a opção 4;
5. Obter uma ordenação dos maiores Compradores desde sempre, seleccionando a opção 5;
6. Obter uma ordenação dos maiores Compradores num determinado período de tempo, seleccionando a opção 6;
7. Obter uma ordenação dos maiores Vendedores desde sempre, seleccionando a opção 7;
8. Obter uma ordenação dos maiores Vendedores num determinado período, seleccionando a opção 8;

9. Determinar quanto dinheiro ganhou a **Vintage** em todo o seu funcionamento, selecionando a opção 9;
10. Terminar a sessão e consequentemente gravar o estado do sistema, selecionando a opção 0;

Toda esta informação referida anteriormente pode ser ilustrada através da seguinte figura:

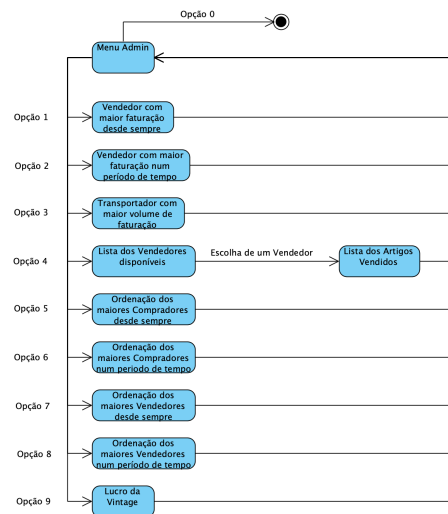


Figura 17: Máquina de Estados do Admin

6 Requisitos

6.1 Requisitos Base de Gestão das Entidades

O nosso programa cumpre a implementação de todos os requisitos base mencionados no enunciado. Permite criar utilizadores (vendedores/compradores), artigos, transportadores e fazer encomendas.

Ao seleccionar **Registar** no Menu Inicial é possível criar CompradorVendedor, bem como, Transportadores. Dentro do Menu de CompradorVendedor, tal como foi explicado anteriormente, é possível criar Artigos e, de igual maneira, fazer encomendas.

6.2 Avançar no Tempo

Quando um Comprador / Vendedor requisita a aplicação para avançar no tempo, tem de fornecer uma data que seja posterior a data atual da aplicação. Mal isto acontece, é efetuada uma comparação das datas de entrega das encomendas que constam na lista de encomendas expedidas, para saber qual delas é que já foram entregues. Para as encomendas que já foram entregues, são removidas da lista de encomendas entregues e adicionadas a lista de encomendas finalizadas. Estas encomendas atualizam o seu estado de "EXPEDIDA" para "FINALIZADA", são atualizadas as listas de artigos a entregar dos Transportadores encargos de entregar os artigos destas encomendas e por fim é também atualizada o valor total de vendas e a lista de artigos vendidos dos vendedores cujos artigos das encomendas lhes pertencessem.

6.3 Efetuar Estatísticas sobre o Estado do Programa

No Menu do Admin é possível ter acesso a um conjunto de estatísticas sobre as compras e vendas que ocorreram na Vintage. A seguir encontra-se uma lista de todas estas estatísticas e a forma como são calculadas.

1. O Vendedor que mais faturou desde sempre, percorrendo a lista de Vendedores e utilizando uma variável temporária para efetuar as comparações entre a faturação dos vendedores.
2. O Vendedor que mais faturou num determinado período de tempo, percorrendo a lista de vendedores a comparando a data das vendas dos produtos com as datas em que a estatística foi pedida, atualizando o contador para depois ser comparado com o contador dos outros vendedores, sendo atualizado se necessário a cada iteração.
3. O Transportador com maior volume de faturação, sendo calculado da mesma forma que a estatística 1, só que percorrendo as listas de transportadores.
4. Listar os Artigos vendidos por um Vendedor, sendo feito uma procura pela lista de artigos vendidos do vendedor pretendido.
5. Obter uma ordenação dos maiores Compradores desde sempre, sendo utilizado o Comparador "ComparadorComprador" para construir um TreeSet que ordene os Compradores por valor total de compras, sendo apenas preciso percorrer esse TreeSet e mostrar os resultados.
6. Obter uma ordenação dos maiores Compradores num determinado período de tempo, sendo utilizado o Comparador "ComparadorCompradorTempo" para construir um TreeSet que ordene os Compradores por valor total de compras dentro das datas pedidas na estatística, sendo apenas preciso percorrer esse TreeSet e mostrar os resultados.
7. Obter uma ordenação dos maiores Vendedores desde sempre, sendo utilizado o Comparador "ComparadorVendedor" para construir um TreeSet que ordene os Vendedores por valor total de vendas, sendo apenas preciso percorrer esse TreeSet e mostrar os resultados.

-
8. Obter uma ordenação dos maiores Vendedores num determinado período de tempo, sendo utilizado o Comparador "ComparadorVendedorTempo" para construir um TreeSet que ordene os Vendedores por valor total de vendas dentro das datas pedidas na estatística, sendo apenas preciso percorrer esse TreeSet e mostrar os resultados.

6.4 Alterar os Transportadores e prever a noção de Premium

O nosso programa permite um Transportador alterar os valores de cálculo, tais como, **Valor Base Pequena, Valor Base Média, Valor Base Grande, Margem Lucrativa do Transportador e Imposto**. Para isso, o *user* insere um valor e simplesmente damos *set* na variável respetiva e, posteriormente, atualizamos o Transportador.

Igualmente, permite o utilizador mudar a sua fórmula de cálculo na integridade, para isso, usamos a biblioteca **ExpressionBuilder** para criar uma expressão matemática personalizada com base na nova fórmula inserida pelo *user*. O processo começa com a leitura da fórmula através da entrada do utilizador. Em seguida, é verificado se a fórmula contém apenas caracteres permitidos, como números, operações aritméticas básicas e as variáveis **VB, MLT e I**. Se a fórmula for válida, os valores de **VB Grande, VB Média, VB Pequena, MLT e I** são solicitados ao *user* e atualizados ao Transportador correspondente.

Em seguida, a expressão é avaliada através do método *evaluate()* e é capturada uma possível exceção caso ocorra algum erro durante a avaliação. No final, a fórmula é atualizada para o transportador correspondente.

Por fim, na adição de um Artigo à venda é questionada se o Artigo é *premium* ou não e caso seja adicionadas as perguntas relativas. Por outro lado, na criação de uma Transportadora é questionado também se esta é *premium* ou não.

6.5 Automatizar a Simulação

Por uma questão de falta de tempo devido à quantidade de cadeiras por parte dos elementos do grupo não cumprimos esta funcionalidade.

7 Conclusão

Neste trabalho, tivemos a oportunidade de explorar o paradigma da programação orientada a objetos de uma maneira prática e aplicada. Fomos apresentados aos novos padrões de desenvolvimento de aplicações nesse paradigma e aprofundamos o nosso conhecimento quanto aos conceitos fundamentais, como encapsulamento, modularidade e abstração de dados. Desta forma, podemos adicionar ou modificar funcionalidades de forma isolada, preservando a integridade e a estabilidade do código já implementado. Essa abordagem também nos permite reutilizar componentes e adaptar o sistema de acordo com as necessidades do projeto, resultando num programa mais escalável, eficiente e de fácil manutenção.

Aprendemos que um dos elementos-chave para garantir a longevidade de uma aplicação é dedicar tempo ao planejamento adequado, embora isso possa surgir enquanto um desafio em certas situações. Ao elaborar o esboço inicial do diagrama de classes e, posteriormente, o diagrama final, foi possível compreender a importância de certos elementos do Java, como classes abstratas, interfaces e estruturas de dados.

Em suma, fazemos uma análise extremamente positiva do trabalho realizado. Reconhecemos que existem áreas que podem ser aprimoradas, no entanto, acreditamos que o trabalho árduo e o esforço empregado valeram a pena, devido ao conhecimento valioso adquirido ao longo do processo.

8 Referências Bibliográficas / Websites

1. Java Documentation

<https://docs.oracle.com/en/java/javase/20/docs/api/index.html>

2. StackOverFlow

<https://stackoverflow.com/>

3. GueeksForGeeks

<https://www.geeksforgeeks.org/>